Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №4
по курсу «Логика и основы алгоритмизации в инженерных задачах»
на тему «Бинарное дерево поиска»

Выполнил:
студент группы 25ВВВ2
Чураева А.А.
Доблер Б.С.

Приняли:
к.т.н., доцент Юрова О.В.
к.т.н., доцент Деев М.В.

Пенза 2026

Задание

1. Реализовать алгоритм поиска вводимого с клавиатуры значения в уже созданном дереве.
2. Реализовать функцию подсчёта числа вхождений заданного элемента в дерево.
3. * Изменить функцию добавления элементов для исключения добавления одинаковых символов.
4. * Оценить сложность процедуры поиска по значению в бинарном дереве.

Практическая часть

Псевдокод

Общие операции для всех структур

Создание нового элемента (get_struct)

```
ФУНКЦИЯ get_struct(data):
  ВЫДЕЛИТЬ память под узел r
  ЕСЛИ r = NULL ТО
    ВЫВЕСТИ "Ошибка выделения памяти"
    ЗАВЕРШИТЬ программу с кодом 0
  КОНЕЦ ЕСЛИ
  r.left  = NULL
  r.right = NULL
  r.data  = data
  ВЕРНУТЬ r
КОНЕЦ ФУНКЦИИ
```

Построение дерева с дубликатами (CreateTree)

```
ФУНКЦИЯ CreateTree(root, r, data):
  ЕСЛИ r = NULL ТО
    r = get_struct(data)
    ЕСЛИ root = NULL ТО
      ВЕРНУТЬ r
    КОНЕЦ ЕСЛИ
    ЕСЛИ data > root.data ТО
      root.left = r
```

```
    ИНАЧЕ
      root.right = r
    КОНЕЦ ЕСЛИ
    ВЕРНУТЬ r
  КОНЕЦ ЕСЛИ
  ЕСЛИ data > r.data ТО
    CreateTree(r, r.left, data)
  ИНАЧЕ
    CreateTree(r, r.right, data)
  КОНЕЦ ЕСЛИ
  ВЕРНУТЬ root
КОНЕЦ ФУНКЦИИ
```

Построение дерева без дубликатов (CreateTreeUnique)

```
ФУНКЦИЯ CreateTreeUnique(root, data):
  ЕСЛИ root = NULL ТО
    root = get_struct(data)
    ВЕРНУТЬ root
  КОНЕЦ ЕСЛИ
  ЕСЛИ data < root.data ТО
    root.left = CreateTreeUnique(root.left, data)
  ИНАЧЕ ЕСЛИ data > root.data ТО
    root.right = CreateTreeUnique(root.right, data)
  КОНЕЦ ЕСЛИ
  ВЕРНУТЬ root
КОНЕЦ ФУНКЦИИ
```

Просмотр содержимого (print_tree)

```
ФУНКЦИЯ print_tree(r, l):
  ЕСЛИ r = NULL ТО
    ВЕРНУТЬСЯ
  КОНЕЦ ЕСЛИ
  print_tree(r.right, l + 1)
  ПОВТОРИТЬ l раз: ВЫВЕСТИ "    "
  ВЫВЕСТИ r.data
  print_tree(r.left, l + 1)
КОНЕЦ ФУНКЦИИ
```

Поиск значения (Search)

```
ФУНКЦИЯ Search(r, value):
  ЕСЛИ r = NULL ТО
    ВЕРНУТЬ NULL
  КОНЕЦ ЕСЛИ
  ЕСЛИ r.data = value ТО
    ВЕРНУТЬ r
  КОНЕЦ ЕСЛИ
  ЕСЛИ value < r.data ТО
    ВЕРНУТЬ Search(r.left, value)
  КОНЕЦ ЕСЛИ
  ВЕРНУТЬ Search(r.right, value)
КОНЕЦ ФУНКЦИИ
```

Подсчёт вхождений (CountOccurrences)

```
ФУНКЦИЯ CountOccurrences(r, value):
  ЕСЛИ r = NULL ТО
    ВЕРНУТЬ 0
  КОНЕЦ ЕСЛИ
  count = 0
  ЕСЛИ r.data = value ТО
    count = 1
  КОНЕЦ ЕСЛИ
  ВЕРНУТЬ count + CountOccurrences(r.left, value)
             + CountOccurrences(r.right, value)
КОНЕЦ ФУНКЦИИ
```

Освобождение памяти (FreeTree)

```
ФУНКЦИЯ FreeTree(r):
  ЕСЛИ r = NULL ТО
    ВЕРНУТЬСЯ
  КОНЕЦ ЕСЛИ
  FreeTree(r.left)
  FreeTree(r.right)
  ОСВОБОДИТЬ память r
```

КОНЕЦ ФУНКЦИИ

Главная программа (main)

```
ФУНКЦИЯ main:
  root = NULL
  start = 1
  ВЫВЕСТИ "-1 - окончание построения дерева"
  ПОКА start = 1:
    ВЫВЕСТИ "Введите число: "
    СЧИТАТЬ число D
    ЕСЛИ D = -1 ТО
      start = 0
      ПРЕРВАТЬ цикл
    КОНЕЦ ЕСЛИ
    root = CreateTree(root, root, D)
  КОНЕЦ ПОКА
  ВЫВЕСТИ "Построенное дерево:"
  print_tree(root, 0)
  ВЫВЕСТИ "Введите значение для поиска: "
  СЧИТАТЬ value
  found = Search(root, value)
  ЕСЛИ found ≠ NULL ТО
    ВЫВЕСТИ "Значение ", value, " найдено (адрес: ", found, ")"
  ИНАЧЕ
    ВЫВЕСТИ "Значение ", value, " не найдено"
  КОНЕЦ ЕСЛИ
  cnt = CountOccurrences(root, value)
  ВЫВЕСТИ "Число вхождений значения ", value, ": ", cnt
  FreeTree(root)
  root = NULL
  ВЫВЕСТИ "--- Построение дерева без дубликатов ---"
  ВЫВЕСТИ "-1 - окончание построения дерева"
  start = 1
  ПОКА start = 1:
    ВЫВЕСТИ "Введите число: "
    СЧИТАТЬ число D
    ЕСЛИ D = -1 ТО
      start = 0
```

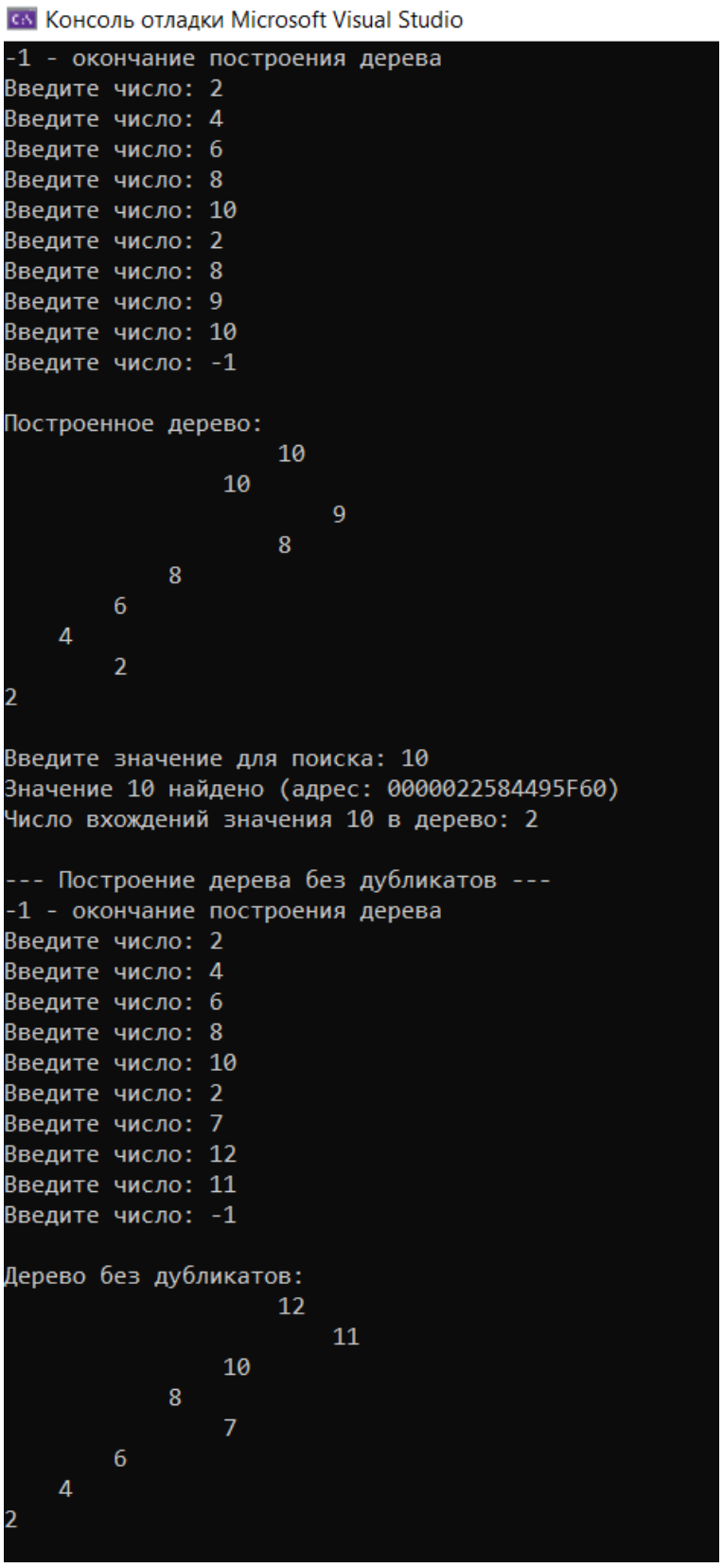
```
        ПРЕРВАТЬ цикл
      КОНЕЦ ЕСЛИ
      root = CreateTreeUnique(root, D)
    КОНЕЦ ПОКА
    ВЫВЕСТИ "Дерево без дубликатов:"
    print_tree(root, 0)
    FreeTree(root)
    ВЕРНУТЬ 0
КОНЕЦ ФУНКЦИИ
```

Сложность поиска в бинарном дереве поиска равна высоте дерева, то есть количеству шагов от корня до самого дальнего листа.

Высота дерева зависит от его формы. В лучшем случае дерево сбалансировано, и при увеличении числа узлов высота растёт очень медленно: чтобы вместить n узлов, нужно удваивать количество узлов на каждом уровне, поэтому высота — это то число, в которое нужно возвести двойку, чтобы получить n. В среднем случае, при случайном порядке ввода данных, дерево получается достаточно ветвистым, и сложность поиска примерно такая же, как в лучшем случае. В худшем случае дерево вырождается в линейный список — это происходит, если данные поступают уже отсортированными.

Бинарное дерево поиска даёт быстрый поиск только тогда, когда оно остаётся сбалансированным. Если данные поступают отсортированными, дерево вырождается, и все преимущества теряются.

Результат работы



Вывод

В ходе лабораторной работы мы научились реализовать алгоритм поиска вводимого с клавиатуры значения в уже созданном дереве. Реализовали функцию подсчёта числа вхождений заданного элемента в дерево. Изменили функцию добавления элементов для исключения добавления одинаковых символов. Смогли оценить сложность процедуры поиска по значению в бинарном дереве.